

COL7(6)83 Assignment 4

Due date: 16 November

This assignment deals with morphological operations, edge detection, and image segmentation. All tasks have to be implemented yourself by working with the array of image pixels, unless otherwise specified in the problem. You are permitted to perform all tasks using intensity images.

Some standard test images used in image processing are provided. You may use these or other images of your own choice for your results.

As with Assignment 2, if you are in a group of two, you are required to complete one of the three optional tasks below; if you are working individually, no optional task is required. In either case, doing one more optional task (beyond the 1 or 0 required) will give some extra marks; doing two more will not increase the marks further.

1. Take a scan of a printed document with lots of lowercase text, such as this example from the textbook (you can use this example itself, for a penalty of 0.5 marks). Crop the image so it contains a similar amount of text as the above example, i.e. about 50-100 words, and make sure the original resolution was sufficiently high that the resulting image is large enough. Threshold and negate it to get a binary image A where the text pixels are 1 and the background is 0.
 - a. Implement binary erosion and dilation using a user-specified structuring element. Demonstrate dilation and closing on A using a moderately sized disk-shaped structuring element. In this part you don't need to show erosion or opening, which may remove all the letters.
 - b. Manually crop a lowercase letter "c" from the image to use as a structuring element B . (You can use an interactive image editing tool for this.) Perform opening with B and show the result. Discuss in your report why not all the c's in the text were found. In this and the following subparts of this question, show the results by creating an RGB image where the red component is A and the green and blue components are the detected letters. Thus, all detected letters will be white and the undetected letters will be red.
 - c. Perform one or more morphological operations on B so that the new SE, B_1 , is able to find all the c's. It will probably also find other letters that are a superset of B_1 , such as some lowercase e's. In your report, show the result and explain what you did to obtain B_1 .

- d. To get only the c's, you will have to use a hit-and-miss transform using B_1 and another structuring element B_2 . Construct B_2 via some morphological operations on B such that you get all the c's in A , and no other letters. Again, explain how you obtained B_2 and show the results.
2. A fundus image is a photograph of the rear surface of the eye, such as [these examples on Wikimedia Commons](#). Choose one of the images on the linked page to use as the input f — don't pick one of the illustrations, pick a real photograph that is sharp and well in focus. The task is to extract the blood vessels in the image and estimate their thickness distribution.
- a. Implement grayscale erosion and dilation using an user-specified *flat* structuring element, and show the results of all four basic operations (erosion, dilation, opening, closing) on f using a moderately sized disk-shaped structuring element.
 - b. Use a bottom-hat transform with a disk-shaped SE to extract the blood vessels in f . Try to choose the radius of the disk so that all the vessels are retained, and other intensity variations are removed. You may need to stretch the contrast of the resulting image to see it better.
 - c. Since the vessels are reflective, some have a bright highlight in the middle (appearing as a dark gap in the bottom-hat transform). This will affect the thickness estimation and so should be removed. Perform one or more grayscale morphological operations to remove the gap while affecting the rest of the vessels as little as possible. Explain what you did in your report.
 - d. Using morphological operations similar to the granulometry technique discussed in class and in the textbook, produce a graph from which one can estimate the typical thickness(es) of the vessels in the image. In your report, discuss your procedure and explain what you conclude about the thickness distribution.
3. Take a photograph of a building having mostly flat walls and straight edges. Identifying these lines and planes can be an important starting point for reconstructing the 3D shape of the building and the camera's position and orientation relative to it.
- a. Implement the Canny edge detection algorithm. You may use built-in library functions for convolution and for morphological reconstruction / hysteresis thresholding. Manually tune the parameters (Gaussian width, high and low thresholds) to find the most important edges in the image, particularly the edges of the building. Show the final result, as well as the intermediate images, namely the gradient magnitude, the result of non-maximum suppression, thresholding at high threshold only, and thresholding at low threshold only.
 - b. Now that the edge pixels have been found, the straight lines need to be identified. Implement the Hough transform and use it to find them. You will need to choose bin

- widths for both ρ and θ ; report the values you used and the size of the resulting bin array. Show the resulting Hough transform image, with intensities scaled to $[0, 255]$.
- c. Find the most important lines in the image by choosing the top k bins in the Hough transform, recovering the lines themselves, and drawing the lines on top of the original image. In case you are getting multiple detections from the same real-life line, non-maximum suppression can be useful here too: start with the bin with the largest value, record it, and zero out nearby bins; then repeat with the largest of the remaining bins. Finally, find all the intersection points of the detected lines that lie inside the image, and mark them on the image as well.
4. While Otsu's method is often successful at automatically determining a reasonable threshold value, it can sometimes choose an obviously wrong threshold, even when the intensities of the foreground and background have no overlap.
- a. Consider an $M \times N$ image having pMN foreground pixels and $(1 - p)MN$ background pixels, with their intensities uniformly distributed in $[1 - a, 1 + a]$ for foreground and $[-b, b]$ for background. (For simplicity we're assuming real-valued intensities.) Find the between-class variance as a function of the threshold $k \in [-b, 1 + a]$. Consequently, in what cases will Otsu's method fail to correctly separate the foreground and background?
- b. Suppose we try k-means with $k = 2$, and somehow manage to initialize it with the two cluster means exactly at the true class means, i.e. 0 and 1. What will happen after 1 iteration? Does this agree with your result in (a) or not? Explain the underlying theoretical reasons.
5. Take one of the 200 *colour* training images from the [Berkeley Segmentation Dataset](#). These images are given along with some human-drawn segmentations, which you can use as a guide for the ideal result (though hard to achieve using purely image processing!).
- a. Implement k-means on RGB vectors to classify the colours of all the image pixels, and thereby obtain a segmentation of the image into regions with similar colours. Choose some value of k appropriate for your chosen image. You may wish to initialize the class means by manually picking k points on the image. Show the final segmentation by replacing each pixel's colour with the corresponding class mean, and highlight the boundaries between pixels in different classes.
- b. **Optional:** Extend your k-means implementation to compute superpixels using the SLIC algorithm. Use a value of s between 10 and 20 pixels. You may refer to the original SLIC paper by [Achanta et al. \(2012\)](#) for more details. Visualize the segmentation like in part (a), with each superpixel assigned its mean colour and boundaries between superpixels highlighted.

- c. **Optional:** Implement the graph cut segmentation algorithm of [Boykov and Jolly \(2001\)](#). The input should be a set of known foreground and background pixels (specified e.g. as subrectangles of the image), whose intensity distributions are used to define region costs. Use the built-in maximum flow algorithms in Python ([SciPy's `maximum\_flow`](#)) or Matlab ([`maxflow`](#)) to compute the minimum cut. You will have to convert the $M \times N$ image into a graph with $MN + 2$ vertices, in whatever format is expected by the max-flow algorithm. In case the segmentation is too computationally expensive, you are permitted to resize the image to a small enough size that you can obtain results in under a minute. Visualize the segmentation like in part (a), with the foreground and background in different colours and the cut between them highlighted.
- d. **Optional:** Implement watershed segmentation for intensity images. You may use built-in operations for thresholding, connected component labeling, and other morphological operations. Compute the gradient magnitude of the input colour image (e.g. by taking the RMS gradient magnitude in R, G, and B), and find the watershed segmentation. For best results, you may need to perform some additional filtering before computing watersheds, e.g. first smoothing before taking the gradient, and performing an h-minima transform afterwards. Visualize the segmentation like in part (a), with each basin in a constant colour and the watersheds between them highlighted.

SUBMISSION

You will submit a zip file that contains all your code for the assignment, and a PDF report that includes your results for each of the assignment problems. In the text of the report, also provide all relevant information for each part.

All images should ideally be saved in PNG format, which is lossless and so does not cause any information loss (other than quantization if the intensities are not already in 8-bit format). JPEG is permitted if your submission PDF is becoming too big for the upload limit.

Your assignment should be submitted on [Gradescope](#) before midnight on the due date. There is a separate submission form for the report and the code. Only one person in a group needs to submit. Late days are counted with a quantization of 0.5 days: if you cannot finish the assignment by midnight, get some sleep and submit by noon the following day.

Separately, each of you must individually submit a short response in a separate form regarding how much you and your partner contributed to the work in this assignment. Your assignment marks will not be released until you submit the form.